

UNIVERSITY OF PISA

DEPARTMENT OF COMPUTER SCIENCE

Master in Data Science and Business Informatics

DATA MINING II

Child Mind Institute (CMI) Problematic Internet Use

Report

Students:
Diprima Riccardo (715425)
La Conca Alessia (715358)

Course:
Data Mining

ACADEMIC YEAR 2025/2026

Contents

1 Tabular data **2**

1.1 Data Understanding and Preparation . . . 2
1.1.1 Data Preprocessing . . . 3
1.2 Outlier Detection . . . 4
1.2.1 Dimensionality Reduction and Visualization . . . 5
1.3 Imbalanced Learning . . . 5
1.4 Advanced Classification . . . 6
1.4.1 Logistic Regression . . . 7
1.4.2 Support Vector Machine (SVM) . . . 7
1.4.3 Ensemble Models . . . 8
1.4.4 Gradient Boosting Machines . . . 10
1.4.5 Neural Networks . . . 11
1.4.6 Comparative Summary and Conclusions . . . 12
1.4.7 Explainability . . . 13
1.5 Regression . . . 15
1.5.1 Methods . . . 15
1.5.2 Results . . . 15

2 Time Series **16**

2.1 Dataset Introduction and Preprocessing . . . 16
2.1.1 Data Understanding and Visualization . . . 16
2.1.2 Preprocessing . . . 17
2.2 Motifs and Discords . . . 18
2.2.1 Motif-Based Clustering . . . 19
2.2.2 Medoid Analysis . . . 20
2.2.3 Comparison with Shapelets . . . 21
2.3 Time Series Clustering . . . 21
2.3.1 Visualization . . . 22
2.3.2 Cluster Analysis . . . 23
2.3.3 Patient-Level Clustering . . . 24
2.3.4 Catch22 . . . 25
2.3.5 Conclusions . . . 25
2.4 Time Series Classification . . . 25
2.4.1 Shapelets . . . 26
2.4.2 K-Nearest Neighbors . . . 27
2.4.3 MiniRocket . . . 28
2.4.4 Conclusions . . . 29

1 Tabular data

1.1 Data Understanding and Preparation

The raw dataset contains clinical, demographic, and behavioral records of a cohort of 8,460 participants, described by 82 features (including the target variable). Based on the project's variable dictionary, the variables are classified into the following types:

- **String** (str) (12 variables): Categorical text features, which include the participant identifier (id) and 11 variables representing the season during which different measurements were taken (e.g., Basic_Demos-Enroll_Season, CGAS-Season).
- **Categorical Integer** (categorical int) (31 variables): Integer-encoded categorical variables. These include binary features like the participant's sex (Basic_Demos-Sex) and fitness performance zones, as well as multi-level ordinal ratings like individual items of the Parent-Child Internet Addiction Test (PCIAT) and daily screen time hours (PreInt_EduHx-computerinternet_hoursday).
- **Quantitative Float (float)** (24 variables): Continuous variables representing physical measurements (e.g., height, weight, BMI, body fat percentage) and summary physical activity scores.
- **Quantitative Integer** (int) (14 variables): Discrete numerical variables representing quantitative tests (e.g., CGAS-CGAS_Score, blood pressure values, and raw questionnaire total scores).
- **Target Variable** (sii) (1 variable): The ordinal multi-class target variable (values 0, 1, 2, 3) representing the Severity Impairment Index.

Three of the primary variables of interest in this study are:

1. **Severity Impairment Index (sii)**: The primary ordinal target variable (ranging from 0 to 3) representing the severity of internet addiction, which acts as the benchmark for classification tasks.
2. **Sleep Disturbance Scale (SDS-SDS_Total_Raw)**: The total raw score representing sleep quality. This variable serves a dual role: it is a key clinical feature in the classification task and the primary target variable for the regression models.
3. **Daily Internet Usage (PreInt_EduHx-computerinternet_hoursday)**: A discrete ordinal feature representing daily screen time (from < 1 hour to > 3 hours per day), serving as the primary behavioral exposure variable.

In the data understanding section, we explored the demographic characteristics and clinical features of the participants, focusing on the target variable sii and its relationships with sex, age, and sleep patterns. With the exception of the exclusion of individuals older than 18 years—who were omitted to focus the study on children and adolescents due to the negligible sample size of adult participants—all exploratory analyses in this section are conducted on the raw, unfiltered dataset.

The target variable sii is an ordinal categorical feature representing the severity of internet addiction, derived from the total score of the PCIAT. We analyzed the relative distribution of the SII classes within each sex group (Male vs. Female). As shown in the relative proportion plot (Figure 1), the proportions of mild, moderate, and severe cases are comparable between males and females, though slight variances exist in the absolute counts due to sample sizing. Notably, females present a higher percentage of sii = 0 compared to males.

To investigate age-related trends, participants were grouped into three distinct developmental categories: up to 10 years (children), 10–14 years (pre-adolescence), and 14–18 years (adolescence). The percentage bar plot (Figure 2) reveals that the proportion of moderate and severe internet addiction cases increases in older age cohorts, reflecting the growing autonomy and screen exposure of adolescents.

A key hypothesis is that higher screen exposure negatively impacts health outcomes. We evaluated the relationship between the raw score of the Sleep Disturbance Scale for Children (SDS-SDS_Total_Raw) and the daily internet/computer usage hours (PreInt_EduHx-computerinternet_hoursday). Daily usage is categorized as follows: 0.0 = Less than 1h/day, 1.0 = Around 1h/day, 2.0 = Around 2h/day, 3.0 = More than 3h/day.

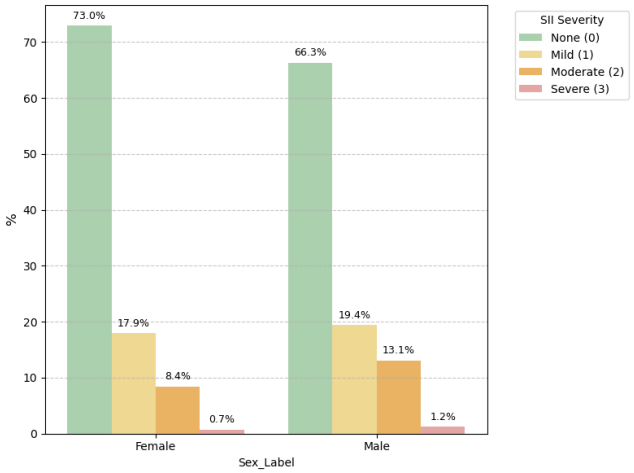


Figure 1: SII distribution by Sex.

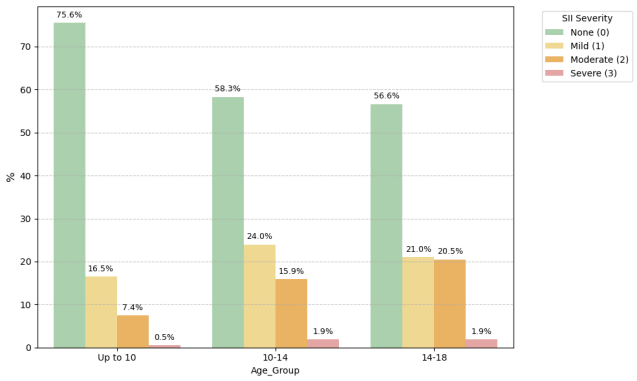


Figure 2: SII distribution across age groups.

A Violin Plot (Figure 3) was chosen to visualize the full probability density distribution and quartiles of the sleep raw score for each screen time group. The visualization clearly shows an upward shift in both median and quartile scores of sleep disturbances as daily internet hours increase, indicating a positive correlation between high screen time and sleep impairment.

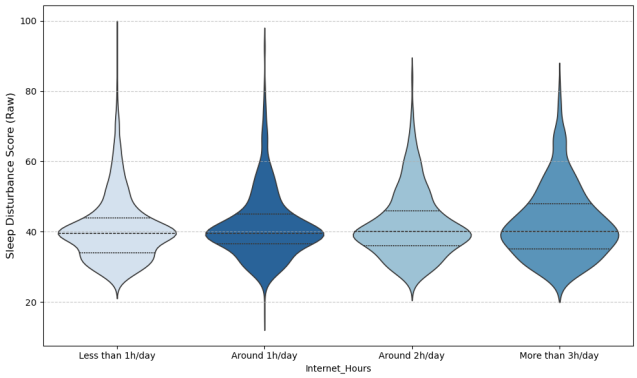


Figure 3: SDS Raw Scores by daily internet usage categories.

1.1.1 Data Preprocessing

The raw dataset contains several missing values, encoding inconsistencies, and out-of-range records. We established a systematic preprocessing procedure to prepare the features. Beginning with the preprocessing phase, a random state of 222 was defined to ensure complete reproducibility; this identical seed was maintained throughout all the analysis involving randomized algorithms.
Several columns contained placeholder or anomalous values that would bias modeling:

1. **CGAS-CGAS_Score**: The score represents global functioning and must strictly range between 1 and 100. Values exceeding 100 were flagged as anomalies and replaced with NaN.

2. **Physical Measurements**: Variables representing physical measurements like weight (Physical-Weight) and blood pressure (Physical-Diastolic_BP, Physical-Systolic_BP) contained impossible zero values. These zeros were replaced with NaN.

3. **BMI**: The variables representing BMI were often inconsistent with each other. For this reason, they were removed, and the variable was calculated manually after the imputation of missing data using height and weight values according to the imperial formula (since height is recorded in inches and weight in pounds): $(\text{Physical-Weight}/\text{Physical-Height}^2) \times 703$.

4. **Fitness_Endurance-Total**: The two variables Fitness_Endurance-Time_Mins and Fitness_Endurance-Time_Sec were merged into a single variable corresponding to their sum, with the time defined in minutes converted into seconds.

5. **PAQ**: The variables concerning the Physical Activity Questionnaire were removed. Most rows showed values in both the adolescent and children versions of the variables. Therefore, they were not considered reliable, and the other physical variables present in the dataset were deemed sufficient for the analysis.

6. **SDS-SDS_Total_T**: This variable was removed because it represents a standardized monotonic transformation of SDS-SDS_Total_Raw. Retaining both would introduce multicollinearity without adding new variance.

7. **Seasons**: Variables related to seasonality were excluded from the dataset to reduce unnecessary dimensionality due to their excessive numbers and similarity.

The final dataset for classification tasks contains 8,226 records.
Missing values in the predictor features were imputed using a multivariate imputer. The participant identifier (id) and the classification target (sii) were removed before this process to completely avoid data leakage. To impute the data, the dataset was split into training set and test set, with a ratio of 70/30. For the sleep scale (SDS-SDS_Total_Raw), we used two different strategies depending on the task:

1. **Classification Task** (sii as target, SDS-SDS_Total_Raw as feature): Since the sleep scale is a feature here, we needed to impute its missing values. To do this safely and avoid leakage, we used a two-stage imputation process. In the first stage, we imputed the other features using IterativeImputer (with a Random Forest estimator) while leaving out the sleep score. In the second stage, we trained a separate Random Forest regressor on these newly imputed features, using only the training rows where the actual sleep score was originally present. This regressor was then used to fill in the missing sleep scores for both the train and test sets. Finally, we rounded the imputed values of categorical and ordinal features to the nearest whole number. Although rounding continuous predictions is not mathematically ideal and can slightly shift class distributions, it was used as a simple way to keep these variables discrete.

2. **Regression Task** (SDS-SDS_Total_Raw as target): We cannot train or test regression models on imputed targets because it would bias our evaluation metrics. Therefore, we created a separate regression dataset by keeping only the rows where the sleep score was originally observed. After removing the outliers, the dataset for the regression task contains 4,787 rows.

1.2 Outlier Detection

The outlier detection phase on the tabular dataset was designed to identify and properly handle the top 1% of anomalous samples. For this phase, the dataset was normalized with a StandardScaler to account for techniques based on distances. To ensure a robust and comprehensive analysis, we selected five different algorithms belonging to distinct families of anomaly detection techniques. Each algorithm was configured to isolate the 1% most extreme observations, either by setting a contamination rate of 0.01 or by extracting the values beyond the 99th percentile of the anomaly scores.
Specifically, for the distance-based family, we implemented a Reverse K-Nearest Neighbors (RKNN) approach. The neighborhood size was set to $k = \sqrt{N} \approx 91$. Instances with an in-degree count at or below the 1st percentile were flagged as outliers.
To represent the deviation-based family, we applied the EllipticEnvelope algorithm. For the ensemble-based approaches, two different models were employed: a Feature Bagging model using Local Outlier Factor (LOF) as the base estimator, and an Extended Isolation Forest (EIF). The Feature Bagging model was built using 100 estimators, with each base LOF model trained on a random subset containing 50% of the features (`max_features` = 0.5).
The Extended Isolation Forest (EIF) was configured with 100 trees, a sub-sample size of 256, and a branching dimensionality set to the full feature size.
To handle high-dimensionality issues, we employed Angle-Based Outlier Detection (ABOD). To avoid the cubic computational complexity, we implemented the fast approximation of ABOD using the k nearest neighbors with $k = \sqrt{N} \approx 91$, which measures the variance of angles between a target point and its neighborhood.
Identifying an observation as an outlier using a single method could lead to a high false-positive rate. To achieve a more reliable identification, we adopted a voting-based strategy where a sample was definitively labeled as an outlier only if it was flagged by at least two out of the five methods.
Regarding the handling of these anomalous records, as the number of detected outliers was exactly 93, we chose to remove them from the dataset entirely instead of treating them as missing values for imputation. By observing the distribution of the outliers in relation to the target variable `sii`, we found that no children with `sii` = 3 were labeled as outliers. The `sii` values for the removed children are 58 with `sii` = 0, 17 equal to 1, and 18 equal to 2. Furthermore, no data point was recognized as an outlier by all 5 models used, while 2 children were detected as

outliers by 4 methods. The final number of 93 outliers (about 1.12% of the dataset) is a natural result of our voting rule. Since we label a point as an outlier if it is flagged by at least two different models, the final combination is slightly different from the individual 1% limit of each algorithm.

1.2.1 Dimensionality Reduction and Visualization

To visually inspect the distribution of the identified outliers, we mapped the high-dimensional pre-processed dataset into lower-dimensional spaces. For this purpose, we applied two distinct dimensionality reduction techniques: Principal Component Analysis (PCA) and t-distributed Stochastic Neighbor Embedding (t-SNE). The goal was to verify whether the points flagged by our voting system exhibited anomalous structural behaviors when projected onto 2D planes.

We first applied PCA, a linear transformation technique that projects the data onto the axes of maximum variance. By extracting the first two principal components, we generated a 2D scatter plot where the instances identified as outliers are highlighted. Because PCA is inherently sensitive to large deviations and extreme values, it provided a very clear validation of our detection methods: the flagged outliers were distinctly positioned at the extreme tails of the principal components, far from the central mass of the dataset. This geometric separation confirms that the removed instances heavily influence the dataset's global variance and behave as global anomalies.

In the resulting 2D t-SNE scatter plot, the normal data points naturally formed high-density clusters. Conversely, the identified outliers were consistently pushed away from these main groups, residing in low-density peripheral regions or appearing scattered. This behavior demonstrates that the detected anomalies are not just extreme in terms of variance, but they are also structurally dissimilar from their closest neighbors in the original high-dimensional space.

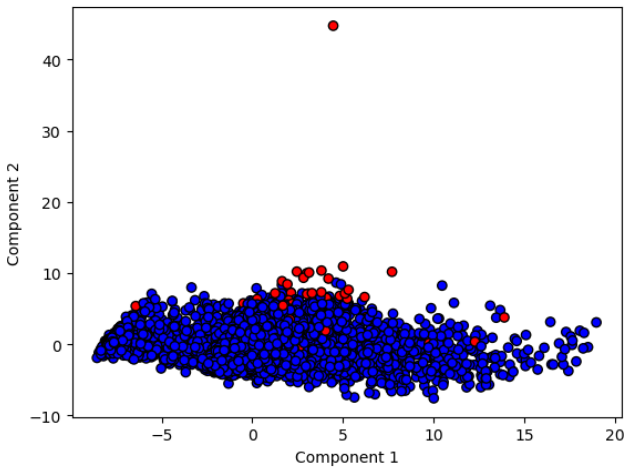


Figure 4: Dimensionality Reduction: PCA

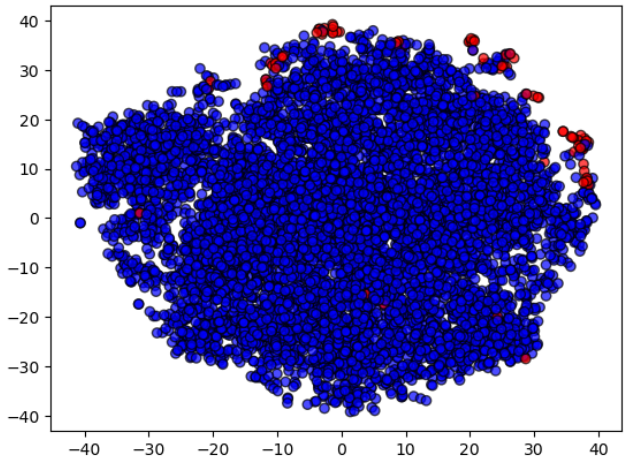


Figure 5: Dimensionality Reduction: t-SNE

The 3D projections for both PCA and t-SNE were also computed and interactively analyzed. These three-dimensional mappings confirmed the findings of the 2D plots, showing even deeper spatial isolation. Specifically, some outliers that appeared to lie near the center of the main clusters in the 2D projection were revealed to be clearly isolated in the outer regions when viewed in the 3D space. However, to maintain visual clarity we chose to include only the 2D representations.

1.3 Imbalanced Learning

Our dataset has a severe class imbalance when predicting the Severity Impairment Index (`sii`). The target variable was binarized by mapping only value 3 to Class 1 (indicating severe risk of internet addiction) and values 0, 1, and 2 to Class 0 (normal, mild, or moderate behavior). Out of 8,226 total samples, 8,141 belong to Class 0 (98.98%) and only 85 belong to Class 1 (1.02%). To avoid target leakage, we completely excluded all individual items of the Parent-Child Internet Addiction Test (`PCIAT-PCIAT_01` to `PCIAT-PCIAT_20`) and the total score (`PCIAT-PCIAT_Total`), as the target variable `sii` is calculated directly from them. This aspect will be furtherly discussed in the Classification section.

We split the dataset into train and test set (70/30 stratified on sii) to apply resampling on the training set and verify the performance on the original test set. We used 5-fold cross-validation to search for the best Decision Tree parameters, optimizing for the minority F1-score. To prevent data leakage, the resampling steps were run inside a pipeline. First, we ran the model on the original imbalanced training set as a Baseline. Hyperparameter tuning chose a depth of 20, but the model mostly predicted the majority class, getting an F1-score of 0.038 (Precision = 0.037, Recall = 0.040) on the test set. For undersampling, we tested Condensed Nearest Neighbour (CNN), which reduced Class 0 to 541 instances. For oversampling, we tested SMOTE and ADASYN, which balanced the training set to 5,701 and 5,714 Class 1 instances, respectively. The parameters and test set results of each classifier are summarized in Table 1.

Table 1: Performance and Hyperparameters of Decision Tree models on Imbalanced Learning task

Method	Criterion	Max Depth	Min Split	Min Leaf	Precision	Recall	F1-Score
Baseline	Entropy	20	2	1	0.037	0.040	0.038
CNN	Gini	3	10	8	0.111	0.041	0.059
ADASYN	Gini	None	10	4	0.021	0.043	0.027
SMOTE	Entropy	20	2	1	0.041	0.120	0.061

- **Baseline**: Trained on the original imbalanced train set. The tree predicts the majority class to maximize accuracy, yielding a recall of 0.040 and F1-score of 0.038.
- **CNN Undersampling**: Reduces the majority class to 541 instances. The cross-validation selected a very shallow tree (`max_depth`=3), which avoids overfitting but fails to identify Class 1 (only 1 True Positive, F1-score = 0.059).
- **ADASYN**: Generates synthetic samples in hard boundary regions. This adaptive method adds noise, making the tree learn artificial patterns and leading to very low precision (0.021) and recall (0.043).
- **SMOTE**: Balances the classes by generating synthetic points. It improves the recall to 0.120 (3 True Positives), but the synthetic interpolation led to many false positives (precision = 0.041).

We also calculated the ROC curves for all models (Figure 6). The model with the highest AUC is SMOTE (0.635). SMOTE achieves the best recall (0.120), showing that oversampling forces the tree to split on Class 1, though with a low precision (0.041) because of the synthetic noise. Overall, all balancing techniques perform poorly on the test set, showing that predicting internet addiction without direct survey items is very difficult.

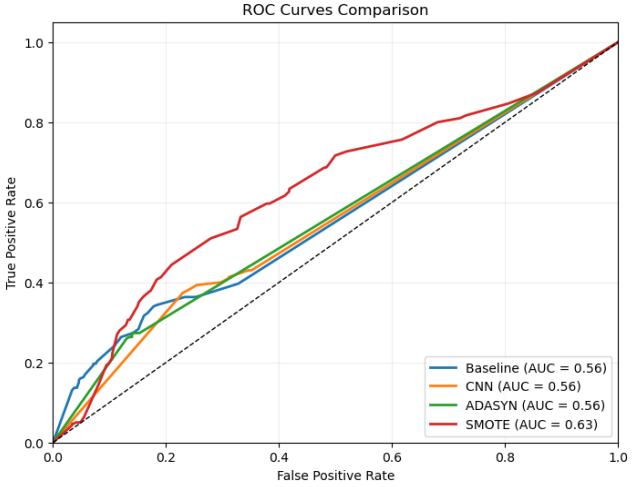


Figure 6: ROC Curve Comparison: Unbalanced Learning

1.4 Advanced Classification

The classification task consists of predicting the target variable sii. The dataset is heavily imbalanced: class 0 accounts for approximately 68% of the training set, while class 3 represents less than 1%.

In this phase, we performed a targeted feature selection by removing specific variables from the dataset. We chose to remove all the questionnaire variables (PCIAT) that are used to directly compute the sii. Two main reasons dictated this decision. On one hand, the target variable is directly derived from PCIAT_Total, which in turn is the sum of the other individual variables. Providing such features would introduce severe data leakage, giving the classifier the exact information needed to trivially compute the correct class. On the other hand, the core aim of the study is to predict problematic internet usage based on physical activity, fitness, metabolic data, and other physical variables. Relying on direct psychological questionnaires would completely bypass this business objective. Although this strategic removal naturally led to lower overall model performance, it was necessary. It forces the machine learning algorithms to investigate the true underlying relationships between problematic internet use and the remaining clinical, demographic, and physical characteristics.
The resulting dataset was then partitioned by assigning 70% of the observations to the training set and the remaining 30% to the test set. Afterwards, to facilitate numerical stability and optimal weight convergence, the input features were normalized using a StandardScaler. To handle the severe class imbalance, we adopted two different strategies depending on the model architecture, keeping in mind the computational complexity. Initially, applying synthetic oversampling techniques to all models proved to be computationally prohibitive and provided minimal improvement. For this reason, we decided to prioritize the use of the native class_weight parameter in models that support it.
For models that do not support class weights (AdaBoost, Gradient Boosting, and XGBoost), we compared training them with and without imbalance correction. For each of these techniques, we chose SMOTETomek which combines oversampling and undersampling to handle the class imbalance. To tune the hyperparameters, a Stratified 10-fold cross-validation optimized for the F1-macro metric was used.

1.4.1 Logistic Regression

A Multinomial Logistic Regression model was implemented and evaluated. To address the severe class imbalance inherent in the dataset, the objective function incorporated balanced class weights (class_weight='balanced'). Regularization was enforced using an L_2 (Ridge) penalty term combined with the lbfgs optimization algorithm. The hyperparameter tuning phase utilized a 10-fold Grid Search Cross-Validation targeting the maximization of the macro-averaged F_1-score to optimize C over the range ($[10^{-4}, 10^{2}]$). During the evaluation phase on the test set, the optimal logistic regression estimator achieved an overall accuracy of 0.49 and a macro-averaged F_1-score of 0.33, as shown in table 2. While it demonstrated reasonable performance on the majority class (Class 0) with an F_1-score of 0.67, its predictive power deteriorated heavily on the minority classes, dropping to an F_1-score of 0.04 for Class 3. Due to the class_weight='balanced' natively parameter, the model achieves the highest recall value for class 3 out of every model, penalizing at the same time the other classes. This performance discrepancy suggests that the linear decision boundaries of logistic regression are insufficient to fully capture the complex, non-linear relationships within the feature space.

Table 2: Classification report: Logistic Regression.

Class	Precision	Recall	F1-score
0	0.83	0.56	0.67
1	0.31	0.30	0.31
2	0.26	0.37	0.31
3	0.02	0.42	0.04

1.4.2 Support Vector Machine (SVM)

SVM searches for a separating hyperplane by mapping input features to a high-dimensional space. We tuned its hyperparameter space using RandomizedSearchCV over the following parameters: kernel in ['rbf', 'poly', 'linear'], C in [0.001, 0.01, 0.1, 1, 10] and gamma in ['scale', 'auto', 0.001, 0.01, 0.1, 1.0]. The model handled class imbalance via native class weighting, setting class_weight='balanced'.
The best parameters found were C=1, gamma='scale', kernel='rbf', yielding a CV F1-macro of 0.3218.

Table 3: SVM Classification Report

Class	Precision	Recall	F1-score
0	0.82	0.57	0.67
1	0.30	0.43	0.35
2	0.25	0.40	0.31
3	0.01	0.08	0.02

SVM shows a moderate ability to spread its predictions across the minority classes, as evidenced by recalls of 0.43 for class 1 and 0.40 for class 2. However, this comes at the expense of precision (0.30 and 0.25, respectively), resulting in a high rate of false positives. The severe class 3 remains virtually unclassifiable (F1-score 0.02, precision 0.01). To better understand these results, Figure 7 shows the SVM decision boundaries on the first two PCA components. We can see a lot of overlap between the classes in the main cluster on the left, which makes it very hard for the model to separate them. Also, the RBF kernel creates small isolated regions to classify the minority class points on the right. This happens because we used balanced class weights to help the model predict the minority classes.

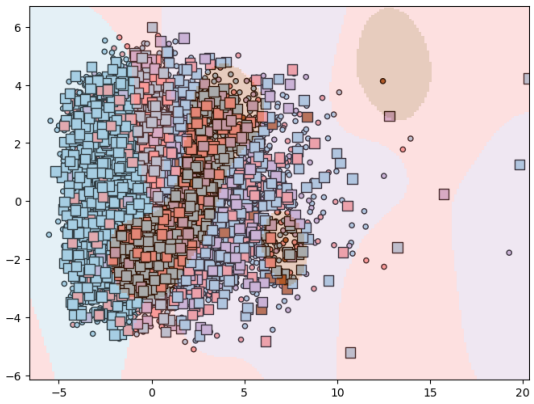


Figure 7: SVM Decision Boundaries with rbf Kernel (PCA components)

1.4.3 Ensemble Models

Random Forest Random Forest is a bagging ensemble method that trains decision trees on bootstrap samples and averages their votes. Trees were allowed to grow fully (max_depth=None). The search was performed using RandomizedSearchCV over: n_estimators in [100, 150, 200, 250, 300], min_samples_split in [2, 5, 8, 9, 10, 11, 12, 15], min_samples_leaf in [1, 2, 3, 4, 8], criterion in ['gini', 'entropy'], max_features in ['sqrt', 'log2'], and class_weight in ['balanced', 'balanced_subsample'].
The best parameters were n_estimators=300, min_samples_split=5, min_samples_leaf=8, max_features='sqrt', criterion='entropy', class_weight='balanced_subsample', achieving a CV F1-macro of 0.3525.

Figure 8: Feature Importance Random Forest

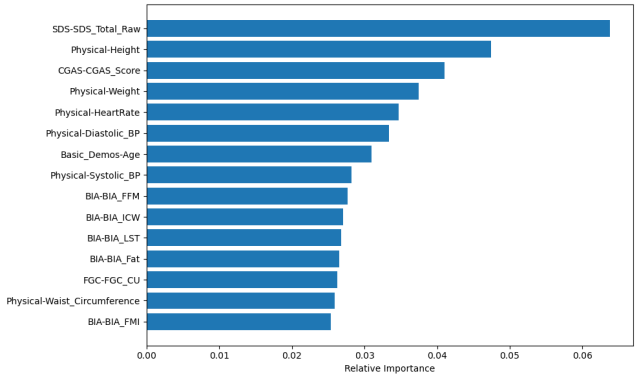


Table 4: Random Forest Classification Report

Class	Precision	Recall	F1-score
0	0.77	0.76	0.77
1	0.31	0.27	0.29
2	0.30	0.36	0.32
3	0.03	0.04	0.03

The feature importance of the Random Forest, shows that the most relevant is SDS-SDS_Total_Raw, which is related to sleep disturbances. This indicates that sleep quality is strongly connected to problematic internet use.

Bagging Classifier The Bagging Classifier trains base decision trees on bootstrap subsets of rows and columns. We ran RandomizedSearchCV over: n_estimators in [50, 100, 150], max_samples in [0.7, 0.8, 0.9], and max_features in [0.5, 0.8, 0.9]. Additionally, to prevent the trees from growing to full purity, we tuned the base estimator's pre-pruning parameters, specifically min_samples_split in [5, 10, 15] and min_samples_leaf in [2, 5, 8, 10]. The base estimator used class_weight='balanced' to manage imbalance.
The best parameters were n_estimators=100, max_samples=0.8, max_features=0.9, estimator__min_samples_split=5, and estimator__min_samples_leaf=8, yielding a CV F1-macro of 0.3740.

Table 5: Bagging Classifier Classification Report

Class	Precision	Recall	F1-score
0	0.78	0.80	0.79
1	0.30	0.28	0.29
2	0.25	0.26	0.25
3	0.33	0.04	0.07

The model demonstrates a balanced behavior across the main classes, showing a recall of 0.80 for the majority class 0, and recalls of 0.28 and 0.26 for the minority classes 1 and 2, respectively. The introduction of pre-pruning via min_samples_leaf=8 prevents the base decision trees from reaching 100% purity. However, class 3 remains poorly predicted with a recall of 0.04.

AdaBoost AdaBoost trains sequential weak estimators using the multi-class SAMME algorithm, adjusting sample weights based on classification errors. To address imbalance, we compared training with SMOTETomek in-fold against a baseline without resampling. We performed a GridSearchCV over: n_estimators in [100, 200, 300], learning_rate in [0.1, 0.5, 1.0], and base estimator max_depth in [1, 2].
For the resampled model, the optimal parameters were n_estimators=200, learning_rate=0.5, max_depth=2, with a CV F1-macro of 0.3313. For the unresampled model, the optimal parameters were n_estimators=300, learning_rate=1.0, max_depth=2, yielding a CV F1-macro of 0.3071.

Table 6: AdaBoost Classification Report: SMOTETomek Resampled (Left) vs. Unresampled (Right).

Class	Precision	Recall	F1-score
0	0.75	0.61	0.68
1	0.26	0.36	0.30
2	0.26	0.26	0.26
3	0.02	0.12	0.03

Class	Precision	Recall	F1-score
0	0.73	0.90	0.81
1	0.32	0.16	0.21
2	0.33	0.16	0.22
3	0.00	0.00	0.00

The combination of SMOTETomek and shallow trees (depth=2) enabled AdaBoost to better capture the minority classes, raising recalls to 0.36 (class 1), 0.26 (class 2) and 0.12 (class 3). Without resampling, the model is highly biased towards the majority class (recall 0.90) and completely fails to predict class 3 (recall 0.00). However, the synthetic oversampling introduces noise that degrades precision across all classes.

1.4.4 Gradient Boosting Machines

Gradient Boosting Gradient Boosting builds sequential decision trees to fit the pseudo-residuals of the gradient of the loss function. We performed a RandomizedSearchCV over: n_estimators in [50, 100, 150], learning_rate in [0.05, 0.1, 0.2], max_leaf_nodes in [4, 8], subsample in [0.7, 0.9], and max_features in ['sqrt', 0.8].
Trained with SMOTETomek, the best parameters were n_estimators=50, learning_rate=0.2, max_leaf_nodes=8, subsample=0.7, max_features='sqrt', yielding a CV F1-macro of 0.3393. Without resampling, the optimal parameters were n_estimators=150, learning_rate=0.2, max_leaf_nodes=8, subsample=0.9, max_features=0.8, yielding a CV F1-macro of 0.3111.

Table 7: Gradient Boosting Classification Report: SMOTETomek Resampled (Left) vs. Unresampled (Right).

Class	Precision	Recall	F1-score
0	0.74	0.76	0.75
1	0.29	0.21	0.25
2	0.31	0.33	0.32
3	0.02	0.08	0.04

Class	Precision	Recall	F1-score
0	0.73	0.92	0.82
1	0.33	0.14	0.20
2	0.40	0.17	0.24
3	0.00	0.00	0.00

Under SMOTETomek, Gradient Boosting achieves higher recalls on the minority classes (notably 0.33 for class 2 and 0.08 for class 3), preventing the model from collapsing. Without resampling, the unregularized model targets the majority class (recall 0.92) and displays a complete inability to predict class 3 (recall and precision at 0.00).

XGBoost XGBoost optimizes gradient boosting through L1 and L2 weight regularisation. We ran a RandomizedSearchCV across: n_estimators in [100, 150, 200, 300], max_depth in [2, 3, 4, 5], learning_rate in [0.01, 0.05, 0.1, 0.2], subsample in [0.6, 0.7, 0.8, 0.9], colsample_bytree in [0.5, 0.7, 0.9, 1.0], gamma in [0.1, 0.3, 0.5, 1.0, 2.0], min_child_weight in [3, 5, 10, 20], reg_alpha in [0, 0.1, 0.5, 1.0, 2.0], and reg_lambda in [1.0, 2.0, 5.0, 10.0].
Trained with SMOTETomek, the best parameters were n_estimators=150, learning_rate=0.05, max_depth=2, subsample=0.9, colsample_bytree=0.5, gamma=0.3, min_child_weight=20, reg_alpha=2.0, reg_lambda=1.0, yielding a CV F1-macro of 0.3272. Without resampling, the optimal parameters were n_estimators=300, learning_rate=0.2, max_depth=4, subsample=0.6, colsample_bytree=0.9, gamma=0.5, min_child_weight=3, reg_alpha=0.1, reg_lambda=5.0, yielding a CV F1-macro of 0.3093.

Table 8: XGBoost Classification Report: SMOTETomek Resampled (Left) vs. Unresampled (Right).

Class	Precision	Recall	F1-score
0	0.74	0.70	0.72
1	0.32	0.20	0.24
2	0.29	0.36	0.32
3	0.04	0.31	0.06

Class	Precision	Recall	F1-score
0	0.74	0.91	0.82
1	0.34	0.19	0.24
2	0.42	0.19	0.26
3	0.00	0.00	0.00

With SMOTETomek, XGBoost is the only algorithm that shows some capacity to predict class 3, reaching a recall of 0.31 (though with a very low precision of 0.04, leading to F1=0.06). Without resampling, it fails entirely on class 3 (recall 0.00), demonstrating that weight regularization and shallow trees are insufficient to handle the minority class without synthetic data generation.

LightGBM LightGBM adopts a leaf-wise tree growth strategy. The model handled imbalance via internal class weighting (class_weight='balanced'). We ran RandomizedSearchCV over: n_estimators in [100, 200, 300, 500], learning_rate in [0.01, 0.05, 0.1], num_leaves in [20, 31, 48, 63], max_depth in [-1, 6, 8, 10], min_child_samples in [10, 20, 30, 50], subsample in [0.6, 0.8, 1.0] with bagging_freq=1, colsample_bytree in [0.6, 0.8, 1.0], reg_alpha in [0, 0.01, 0.1, 1.0], and reg_lambda in [0.1, 1.0, 2.0, 5.0].

The best parameters were n_estimators=500, num_leaves=20, max_depth=8, learning_rate=0.01, min_child_samples=20, subsample=0.8, colsample_bytree=0.8, reg_alpha=1.0, reg_lambda=2.0, achieving a CV F1-macro of 0.3585.

Table 9: LightGBM Classification Report (Balanced Class Weights).

Class	Precision	Recall	F1-score
0	0.83	0.65	0.73
1	0.32	0.44	0.37
2	0.28	0.44	0.34
3	0.00	0.00	0.00

LightGBM exhibits strong recall on classes 1 (0.44) and 2 (0.44). Leaf-wise growth led to a F1-macro of 0.3602. However, class 3 remains completely unclassified (recall 0.00), demonstrating the limits of class weights on extremely scarce classes.

1.4.5 Neural Networks

Neural Networks are a powerful tool for modeling complex, non-linear relationships in data, based on a collection of interconnected nodes called artificial neurons. To mitigate the impact of the uneven distribution of labels in y, class balancing was implemented: class weights (class_weight) were calculated on the training set and provided as a hyperparameter to the models' cost function during the learning phase.
The development and training of the Neural Network architectures were conducted utilizing the TensorFlow and Keras frameworks. To systematically identify the most effective hyperparameters, an automated Bayesian optimization framework, Optuna, was employed. The tuning process explored both architectural and training parameters, including the number of hidden layers, neurons per layer, activation functions and learning rate. To ensure the robustness of the hyperparameter search, each trial was evaluated through a 5-fold Stratified Cross-Validation. The optimization objective within Optuna was set to maximize the macro-averaged F1-score, to account for the highly imbalanced distribution of the target variable y.
The initial training phase focused on establishing a baseline Deep Neural Network without any explicit regularization techniques. The network weights were optimized over a maximum of 500 epochs monitored by an Early Stopping callback, halting the training process after 20 epochs of no improvement and restoring the weights from the previous optimal epoch. Following the hyperparameter search, the optimal baseline configuration consisted of 3 hidden layers, made of 512, 1024, and 256 neurons respectively, and utilizing the ReLU activation function. The output layer consisted of 4 neurons with a softmax activation,and the training was executed with a learning rate of approximately 0.0024. Afterward, to constrain the network's complexity and prevent the amplification of irrelevant features, weight regularization was introduced by adding penalty terms to the loss function. Two distinct models were trained using L1 (Lasso) and L2 (Ridge) penalties. The Bayesian optimization for the L1 model converged on an architecture with 2 hidden layers (512 and 1024 neurons), utilizing the ReLU activation function, and an optimal regularization factor of $\lambda = 1.61 \times 10^{-5}$. The learning rate was set to approximately 0.0024. On the other hand, the optimal L2 configuration featured 4 hidden layers (2048, 2048, 64, and 64 neurons) with ReLU activations, a regularization factor of $\lambda = 0.0014$, and a learning rate of approximately 0.0021. Both models concluded with a 4-neuron softmax output layer. Finally, as a third approach to structural regularization, a Dropout mechanism was implemented. By randomly deactivating a fraction of the input units at each update during training time, the model is forced to learn more robust features that are not excessively reliant on specific neurons. The optimal configuration identified through the tuning process applied a dropout rate of 15% ($p = 0.15$) after each of the 2 hidden layers, which comprised 512 and 2048 neurons respectively with ReLU activation. The final architecture included a 4-neuron softmax output layer and was optimized using a learning rate of approximately 0.0007.
As seen in table 10, the comparative evaluation of the trained architectures reveals a narrow performance margin between the unregularized baseline and the L2 (Ridge) regularized model, while both the L1 (Lasso) and Dropout configurations underperformed across all tracked metrics. The unregularized model achieved the lowest test loss (0.9951) and the highest overall accuracy (0.6390). However, considering the highly imbalanced nature of the dataset, the macro-averaged F1-score serves as the primary and most reliable evaluation criterion, and on this specific metric the L2 regularized architecture achieved the highest overall score (0.3382), marginally outperforming the Dropout one (0.3376). Given this result, combined with the inherent theoretical advantages of L2 regularization in penalizing extreme parameter values and providing structural robustness against overfitting on unseen data, the L2 (Ridge) model is selected as the optimal architecture for this classification task.

Table 10: Classification reports comparison for all DNN models.

Model	Class	Precision	Recall	F1-score
NOREG	0	0.76	0.81	0.78
	1	0.33	0.28	0.30
	2	0.32	0.30	0.31
	3	0.06	0.08	0.07
L1	0	0.79	0.65	0.71
	1	0.26	0.40	0.32
	2	0.23	0.26	0.25
	3	0.08	0.15	0.11
L2	0	0.77	0.79	0.78
	1	0.31	0.25	0.28
	2	0.29	0.31	0.30
	3	0.10	0.15	0.12
DROPOUT	0	0.79	0.63	0.70
	1	0.29	0.30	0.30
	2	0.23	0.44	0.30
	3	0.03	0.08	0.04

1.4.6 Comparative Summary and Conclusions

Table 11 collects the cross-validation and test set results of all evaluated models.

Table 11: Comparative Classification Results on Target sii.

Algorithm	CV F1-macro	Test Accuracy	Test F1-macro
Logistic Regression	0.3016	0.4935	0.3228
SVM	0.3218	0.5199	0.3398
Random Forest	0.3525	0.6159	0.3528
Bagging	0.3740	0.6367	0.3497
LightGBM	0.3585	0.5818	0.3602
AdaBoost	0.3071	0.6665	0.3087
Gradient Boosting	0.3111	0.6787	0.3148
XGBoost	0.3093	0.6827	0.3304
With Resampling (SMOTETomek)			
AdaBoost	0.3313	0.5199	0.3176
Gradient Boosting	0.3393	0.6029	0.3400
XGBoost	0.3272	0.5624	0.3371
Deep Neural Networks			
DNN (NOREG)	0.3341	0.5770	0.3336
DNN (L1)	0.3322	0.5438	0.3328
DNN (L2)	0.3319	0.5851	0.3382
DNN (DROPOUT)	0.3365	0.5867	0.3376

Comparing the models, we observe that the overall generalisation performance is bounded at a maximum test F1-macro of 0.3602, achieved by LightGBM, closely followed by Random Forest (0.3528). These cost-sensitive models offer the best trade-off, avoiding the generation of synthetic noise and the associated computational overhead of in-fold oversampling.

The introduction of SMOTETomek in-fold improves the CV F1-macro compared to their unresampled counterparts. However, evaluating them on the imbalanced test set reveals a major trade-off: oversampling forces the models to capture minority class instances, which significantly degrades overall test Accuracy. Conversely, unresampled models display high test accuracy but collapse towards the majority class, leaving the rarest classes poorly classified or entirely ignored.

A persistent failure across all models is the inability to correctly predict the severe internet addiction level (Class 3). The Neural Networks are the best at correctly classifying class 3, with a class-specific F1 Macro that reaches

0.12 with the L2 regularized model.
Evaluating the results, it becomes clear there is no strictly better model among the proposed solutions; while the model with the highest F1-Macro is LightGBM, the DNN with L2 regularization is by far the best model at predicting class 3.
In conclusion, attempting to classify the Severity Impairment Index (sii) without using the direct questionnaire variables represents a major limitation. The remaining clinical and physical variables (such as sleep indices, body measurements, and daily functioning scores) correlate with problematic internet use but do not possess enough discriminative power on their own to reconstruct its diagnostic categories. This indicates that while physiological and somatic indicators are valuable, direct behavioral assessments remain indispensable to establish a reliable boundary for internet addiction severity.

1.4.7 Explainability

Explainability refers to the set of techniques designed to render the internal decision-making processes of black-box models transparent and understandable to human users. In this study, post-hoc explainability techniques were applied to the best-performing neural network model, identified during the tuning phase as the L2 (Ridge) regularized architecture. To comprehensively analyze the network's behavior, a dual approach was implemented: SHAP was employed for global explainability, while LIME was utilized for local explainability.
Evaluating the overarching decision-making logic of the neural network first requires a macroscopic perspective. To achieve this, SHapley Additive exPlanations (SHAP) were employed to assess the global feature importance and the directional impact of each variable across the entire dataset. Given the deep learning architecture of the selected model, the DeepExplainer algorithm was utilized. This approach, based on the DeepLIFT framework, allows for an efficient and analytically exact computation of feature attributions by tracing the network's gradients. To establish the expected baseline of the model, a representative dataset of 100 observations was sampled from the training data. Subsequently, SHAP values were calculated for the test set, enabling the aggregation of local attributions into a comprehensive global summary. This macro-level analysis reveals which behavioral, psychological, and anthropometric metrics systematically dictate the network's classification boundaries across all four classes. In figure 9 the SHAP summary plot regarding class 0 is reported as an example. The aggregated results highlight that the most influential feature driving the model's global predictions is the daily computer and internet usage (PreInt_EduHx-computerinternet_hoursday), which exhibits a profound cross-class impact, particularly in distinguishing Class 0 and Class 2. Following closely is the total raw score from the sleep disturbance index (SDS-SDS_Total_Raw), underscoring a strong link between psychological assessments and the network's classification logic. Anthropometric and demographic variables also play a foundational role, with Physical-Height, Basic_Demos-Age and Basic_Demos-Sex ranking, for instance, from fourth to sixth in overall importance for class 0. Interestingly, while physical performance metrics such as FGC-FGC_CU are highly discriminative, as will be noted in the local analysis, their global impact is predominantly concentrated on specific categories, most notably Class 1. Ultimately, the global SHAP analysis demonstrates that the neural network establishes its decision boundaries by heavily prioritizing behavioral habits (screen time) and psychological well-being, supported by structural and demographic baselines.
While SHAP highlights the model's global rules, local explainability is necessary to understand how these rules apply to specific or anomalous cases. To this end, we selected two instances to evaluate with LIME: a correctly classified observation (index 1844) and a misclassified one (index 104). The model assigns the first observation to Class 0 with 60% confidence, leaving 26% for Class 1 and 14% for Class 2. As shown in the LIME explanations, the most influential feature for this prediction is BIA-BIA_Fat ¡= -0.11, providing the largest local contribution (approximately -0.10). Other physical measurements, such as Physical-Height ¡= -0.71 (-0.07) and FGC-FGC_CU ¡= -0.68 (-0.06), also play a key role in the same direction. Conversely, Physical-HeartRate ¿ 0.42 pushes the prediction in the opposite direction with a positive weight of 0.05. Ultimately, this correct classification relies on a consistent pattern of specific physical and structural indicators.
Conversely, the misclassified instance (index 104) reveals a conflicting pattern. Here, the neural network incorrectly predicts Class 2 with 58% confidence, assigning 29% to Class 1 and 12% to Class 0. LIME shows this decision is still heavily driven by BIA-BIA_Fat ¡= -0.11, which strongly pushes the prediction with a weight of -0.095. Other physical metrics, like Physical-Height ¡= -0.71 (-0.06) and FGC-FGC_CU ¡= -0.68 (-0.05), reinforce this direction. Furthermore, the presence of higher cardiovascular indicators, such as Physical-HeartRate ¿ 0.42 (0.05) and Physical-Diastolic_BP ¿ 0.25 (0.04), complicates the model's logic by providing positive contributions. This highlights a vulnerability in the classifier: conflicting signals between structural metrics and cardiovascular responses can create ambiguity, ultimately leading to a misclassification.

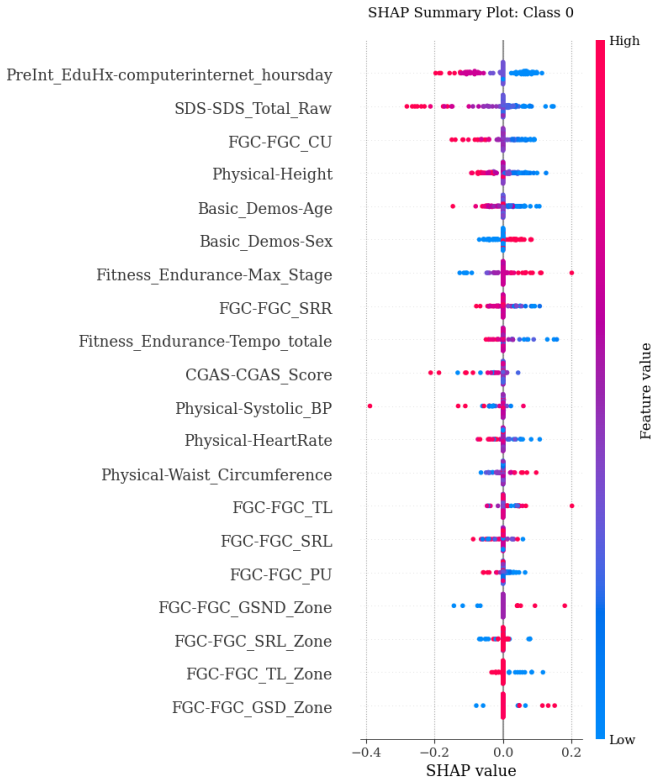


Figure 9: SHAP visualization

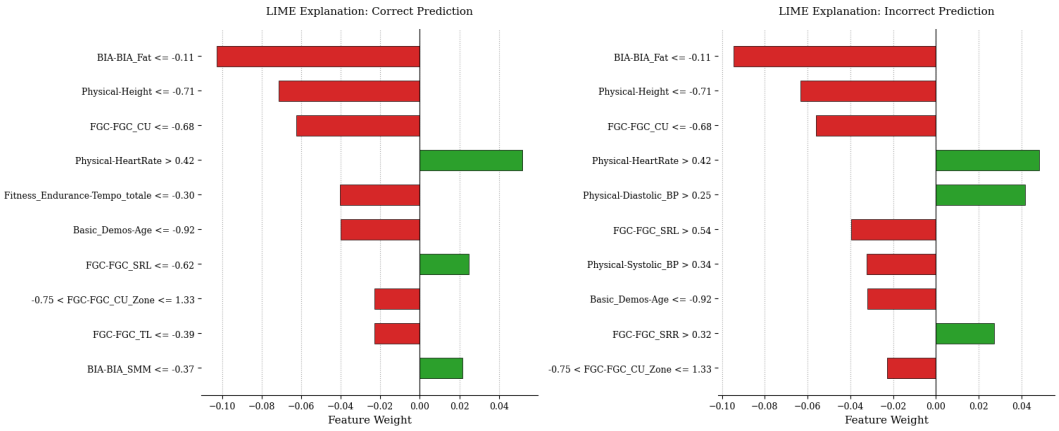


Figure 10: LIME visualization

1.5 Regression

A multiple regression task was defined with the aim of studying the variable "SDS-SDS_Total_Raw", the raw values of the sleep disturbance scale. It was chosen because, among the few continuous variables in the dataset, it represents a physiological indicator connected with internet usage and daily routines. Since sleep quality might be compromised by excessive screen time, looking for the factors that predict sleep disturbances can offer valuable secondary insights into the subject's overall well-being, complementing the previous classification analysis of the previous chapters. Furthermore, a multivariate non-linear regression task is able to uncover the complex relationships between the subject's sleep profiles and physiological, psychological, and metabolic profiles.

1.5.1 Methods

To prepare the dataset for this task, the variables Id and SDS-SDS_Total_T and sii were removed. Subsequently, the target variable was separated from the regressors, and the dataset was partitioned into training and test sets using a 70-30 split ratio. The models selected to perform the regression task are LightGBM, Support Vector Regression (SVR), and a Bagging-based ensemble method.

LightGBM was chosen as the primary gradient boosting framework due to its computational efficiency and its capability to model complex non-linear interactions through a leaf-wise tree growth strategy. The hyperparameters were chosen by executing a grid search with 5-fold cross-validation; the values were used as follows: learning_rate = 0.05, n_estimators = 50,num_leaves = 31. The optimal model configuration was trained on the train_set and subsequently evaluated on the test set. The performance assessment returned a Mean Absolute Error (MAE) of 6.514, a Mean Squared Error (MSE) of 78.949, and an R^2 score of 0.110.

Support Vector Regression (SVR) was implemented to evaluate the effectiveness of margin-based optimization for this regression task. Given the high sensitivity of this algorithm to the scale of input features, a Standard-Scaler was applied to normalize the data prior to model training. The optimal hyperparameters were identified by executing a grid search with 5-fold cross-validation, resulting in the following configuration: kernel='rbf', C=1.0, epsilon=0.01, and gamma=0.01. This model was trained on the training set and subsequently evaluated on the test set, resulting in a Mean Absolute Error (MAE) of 6.428, a Mean Squared Error (MSE) of 84.823, and an R^2 score of 0.044.
A Bagging ensemble approach was utilized to reduce model variance and prevent overfitting by training multiple base estimators on random subsets of the dataset. The basic estimator used was a DecisionTreeRegressor. Consistent with the methodology applied to the other models, the hyperparameters were chosen by executing a grid search with 5-fold cross-validation. The optimal configuration identified was n_estimators=200, max_samples=0.7, and max_features=0.7. Following this, the model was trained on the training set and subsequently evaluated on the test set. The performance assessment returned a Mean Absolute Error (MAE) of 6.528, a Mean Squared Error (MSE) of 78.761, and an R^2 score of 0.111, representing the best overall performance among the tested models.

1.5.2 Results

Comparing the results obtained from the three non-linear models,as shown in table 12, the Bagging model achieved the best overall performance, and LightGBM closely followed after. The Bagging Regressor demonstrated the lowest prediction errors (MAE and MSE) and the highest proportion of explained variance (R^2). LightGBM closely followed, yielding a MAE of 6.51 and an R^2 of 0.110. Conversely, Support Vector Regression (SVR) reported the worst results out of the three, with a MAE of 6.428 and an R^2 score of 0.044.

The most important finding across all experiments is the quite low coefficient of determination (R^2), which never got over 12% in any configuration of hyperparameters and seed studied. This implies that roughly 88% of the total variance in the SDS-SDS_Total_Raw target variable remains unexplained by the chosen regressors. This significant limitation is not due to any problem related to the chosen algorithms, which were properly tuned and validated, but rather originates from the limitations of the available feature space. Sleep disturbances in children and adolescents constitute a highly complex, biological and behavioral process, and it seems now clear that the variables included in this dataset were not particularly connected to the y, and do not capture the direct and fundamental determinants of sleep quality. The low predictive capability highlights that physical activity and metabolic profiles, while tied to general systemic health, offer only a partial and indirect reflection of sleep disturbances. The Bagging and LightGBM models proved marginally better at handling the high level of noise and weak non-linear relationships within the data compared to SVR.

Table 12: Regression Results

Modello	**MAE**	**MSE**	R^2
LightGBM	6.514070	78.948857	0.109850
SVR	6.427632	84.822635	0.043623
Bagging	6.527791	78.761096	0.111967

2 Time Series

2.1 Dataset Introduction and Preprocessing

The second part of the project focuses on the study of a dataset comprising 4437 time series, each consisting of 200 time steps and 13 attributes. Each time series contains data regarding a subject from the study; the data was collected through the use of a wrist-worn sensor, and every step represents the average value recorded over a 30-minute interval.

2.1.1 Data Understanding and Visualization

At first, some randomly chosen time series were explored through graphics and metrics (measures of central tendency and variability). It was noted that certain variables remained static across all the 200 steps; specifically, it was confirmed that the attributes weekday, quarter, relative_date_PCIAT, id, and sii_binary were temporally constant within each subject's respective time series. The variables included are the following:

- **Movement and sensor related variables**:
  - **X, Y, Z**: Measurements of acceleration along the x, y, and z axes, respectively.
  - **enmo**: Euclidean Norm Minus One, a standard metric used to quantify physical activity intensity by calculating the vector magnitude of acceleration minus gravity.
  - **anglez**: The angle of the z-axis relative to the horizontal plane, typically used to infer arm posture and assist in detecting sleep/wake states.
  - **non_wear_flag**: A binary indicator specifying whether the wrist-worn device was estimated to be removed by the subject during the recorded time step.
  - **light**: The ambient light level recorded by the device's sensor.
  - **battery_voltage**: The voltage level of the device's battery during the recording period.
- **Temporally constant variables**:
  - **weekday**: An indicator of the day of the week corresponding to the recorded time step.
  - **quarter**: An indicator representing the quarter of the year in which the recording took place.
  - **relative_date_PCIAT**: The date of the sensor recording relative to the administration date of the Parent-Child Internet Addiction Test (PCIAT).
  - **id**: A unique identifier assigned to each participant in the study.
  - **sii_binary**: The binarized target variable Sii.

Another interesting finding was the fact that some participants in the study were associated with multiple time series. In table 13 are reported the 5 five patients with most time series in the dataset. The overall number of patients present in the dataset is 494, which implies that, on average, each participant is associated with ≈ 9 time series.

Table 13: Top 5 most recurrent childen

Id	Series	sii
2043	36	0
1935	35	0
210	34	0
2712	34	0
1328	33	0

Figure 11 illustrates the distributions of the 13 variables. Although Y, Z, and anglez approximate a normal distribution, the X axis exhibits a pronounced bimodal pattern. Given the assumption that kinematic variables typically follow a normal distribution, we hypothesized the presence of two overlapping normal distributions within the X data. To mathematically isolate these overlapping states, a Gaussian Mixture Model (GMM) was applied, and the means of the two underlying normal distributions were identified at -0.5253 and 0.4364. Furthermore, variables such as enmo, light, and relative_date_PCIAT are heavily right-skewed, displaying long exponential tails with values concentrated near zero. In contrast, battery_voltage demonstrates a left-skewed distribution. The categorical features non-wear_flag and sii_binary are strictly binary, with the former being severely imbalanced. Finally, the temporal indicators weekday and quarter present discrete, roughly uniform distributions, whereas the id variable shows the irregular frequency spread inherent to subject identifiers. Although the variables Z and anglez exhibited a strong correlation, both were retained in the following analysis as they capture distinct physical aspects: movement and inclination.

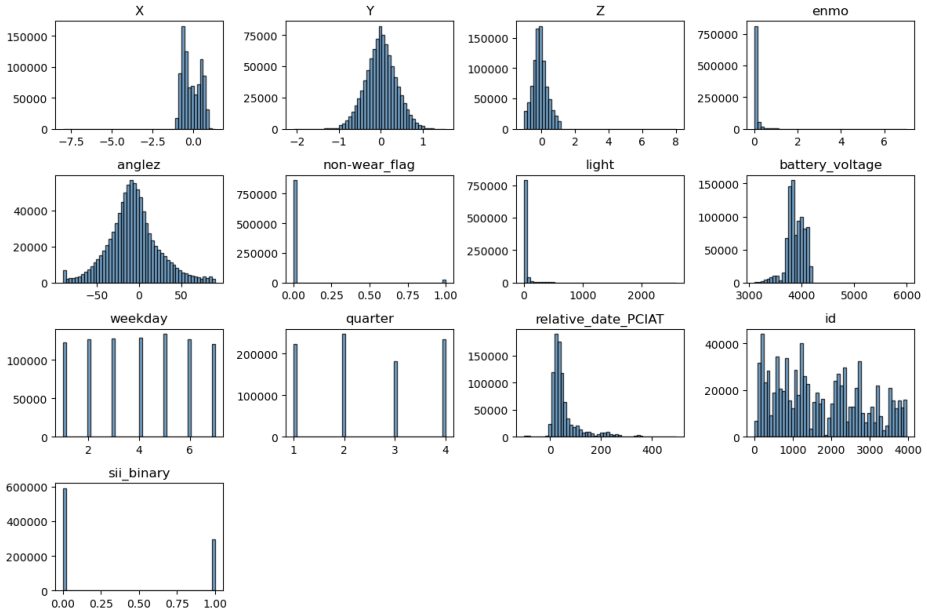


Figure 11: Global Variable Distribution

2.1.2 Preprocessing

Subsequently, a research for missing values was performed, confirming that there were no explicit missing values across any of the observations. We did notice, however, that approximately 20000 time steps (around 2.2% of the entire dataset) had the variable non_wear_flag=1. During these specific intervals, the sensor was presumed to not be worn by the users and would report readings indicating stillness. Given that for many of the affected time series the sensor was removed for the majority of the observed period, we opted against removing and imputing back these values; introducing artificial data over long gaps would not accurately reflect the subject true behavior pattern, therefore the original values were retained.

To handle anomalous observations, an outlier detection and removal process was executed solely on the light variable, which is highly prone to sensor errors and non-systematic spikes. Physical activity signals (e.g., enmo and

raw accelerations) were excluded from this step to prevent the suppression of physical movement peaks, which are highly informative; furthermore, considering that each observation has a duration of 30 minutes, it is less prone to capturing isolated and irrelevant movement spikes. A rolling Hampel filter with a window size of 16 (representing 8-hour intervals) was applied. Although a 6σ threshold deviates from the standard 3σ convention, it was chosen to selectively target and remove only the most extreme spikes, ensuring that typical variations in light exposure were not over-filtered. The lower bound was also clipped to zero to maintain physical plausibility. Observations outside this interval were replaced with NaN values and reconstructed using linear interpolation.
Following outlier removal, the light variable remained heavily right-skewed, with a mean of 53.04 and a standard deviation of 208.23. To achieve a more symmetric distribution and handle zero observations, a $\log(1+x)$ transformation was applied. To extract the underlying signal from the time series, a denoising procedure was implemented. Specifically, a simple moving average (SMA) filter with a rolling window of 5 observations was applied to every variables except the stationary features and the boolean non wear flag (since it would have been pointless to include them). To prevent data loss and the introduction of missing values at the boundaries of the series, the filter was configured to compute the mean using a minimum of one available period. Finally, a normalization process was implemented using a dataset-wise approach to ensure feature comparability. To preserve the relative baseline differences among subjects, such as the inherent distinctions between highly active and predominantly sedentary individuals, a global scaling strategy was adopted. The features were divided into two distinct groups based on the appropriate transformation: Z-score standardization was applied as the primary method, while Min-Max scaling was utilized to ensure better interpretability within a bounded range for specific variables. Specifically, a global Z-score standardization was applied to the spatial and positional variables (X, Y, Z, enmo, and anglez), computing the requisite mean and standard deviation by aggregating the data across all children. Concurrently, a global Min-Max scaler was applied to the battery_voltage and light variables. Lastly, an approximation procedure was employed to create a lighter dataset for future analysis. The Piecewise Aggregate Approximation (PAA) methodology was implemented to reduce the dimensionality of the time series while preserving their fundamental shapes and global trends. Specifically, each normalized time series was divided into 40 equal-sized segments, and the data points within each frame were averaged to represent the segment's value. In figure 12, the final variables distribution is showed.

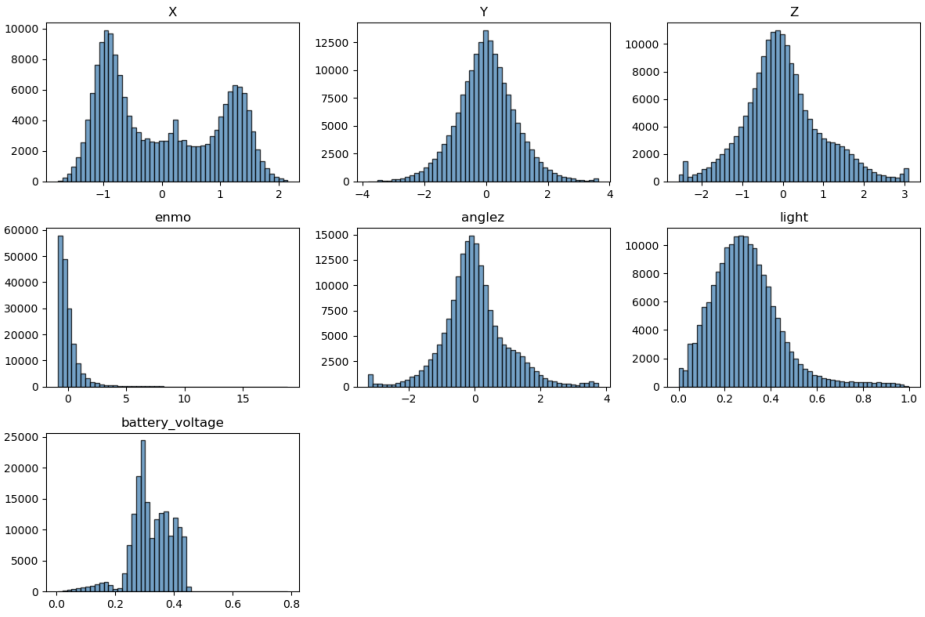


Figure 12: Global Processed Variable Distribution

2.2 Motifs and Discords

To analyze the temporal behavior of subjects and identify characteristic behavioral routines and anomalies, we employ subsequence-based analysis. Specifically, the Matrix Profile framework has been considered, using the STUMP algorithm.

The selected window size is $m = 8$ samples. Given that each sample represents a 30-minute average of the sensor signal, $m = 8$ corresponds to a physical window of 4 hours, to observe temporal phases that represent different parts of the day.
The subsequence analysis was conducted on the enmo variable, which serves as the most representative measure of physical activity. The dataset used was the one to which the dataset-wise normalization described in the pre-processing section had been applied. Furthermore, no approximation technique was applied to the time series in this section.

2.2.1 Motif-Based Clustering

To group subjects based on their most recurrent physical activity routines, we implemented a shape-based motif clustering pipeline. For each of the 4,437 time series, the top-1 motif of length $m = 8$ was extracted from the enmo series. To ensure a stable and comparable scale across the entire cohort during clustering, these motifs were extracted from the normalized dataset. In this analysis, only the physical movement variables (enmo and anglez) and the environmental exposure variable (light) were considered, while other system or administrative variables were discarded.
We clustered the vectors of the enmo motifs using the TimeSeriesKMeans algorithm with the Euclidian distance metric. This groups subjects based on the shape and absolute intensity of their daily routines. The variables other than enmo (specifically, anglez and light) were excluded from the distance computation during the clustering phase. Instead, they were extracted from the original time-series within the same 4-hour motif window to perform a multivariate profiling of the resulting clusters, providing the necessary behavioral and environmental context for each cohort.
The motif-based clustering identified three distinct behavioral routines across the dataset:

- **Cluster 1** Characterized by moderate physical moderately low intensity (mean ENMO = -0.147). Post-clustering profiling reveals moderate postural variations (mean *anglez* $= -0.04°$, standard deviation = 0.46) and low ambient light (mean = 0.30), indicating sedentary routines. This cluster exhibits 33.5% of subjects classified as high-risk ($sii_binary = 1$).

- **Cluster 2:** Dominated by flat, low-intensity movement motifs (mean ENMO = −0.213, standard deviation = 0.388) and minimal light (mean = 0.29). The postural profile exhibits a low mean angle (mean *anglez* = $-0.08°$), but the moderate postural variation (standard deviation = 0.49) indicates micro-adjustments during prolonged resting states or sleep. This cluster shows the highest correlation with sii, at 34.8%.

- **Cluster 3:** Composed of high-intensity active motifs with large physical energy fluctuations (mean ENMO = 1.788, standard deviation = 2.378), high ambient light (mean = 0.44), and standing postures with lower overall variation (mean *anglez* $= 0.07°$, standard deviation = 0.34), reflecting highly active, consistent daytime behaviors. This cluster has the lowest correlation with sii=, at 24.3%.

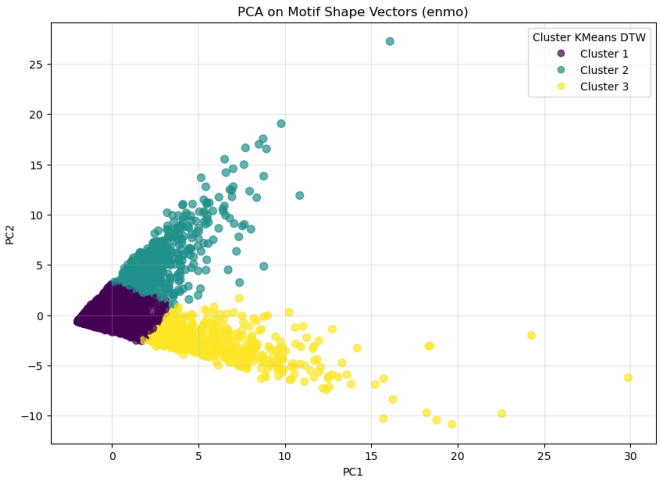


Figure 13: Clustering on Motifs

2.2.2 Medoid Analysis

Since sedentary routines (Cluster 1) represent the direct behavioral context of screen-time and technology abuse, we conducted a motif analysis within this cluster. We extracted two representative medoids (the most central and representative exemplar within the cluster, as it exhibits the minimum cumulative distance to all other cluster members), one for each sii_binary category: Subject 3846 (low-risk, $sii_binary = 0$) and Subject 4386 (high-risk, $sii_binary = 1$).

We set the parameters to extract the top $k = 2$ motifs (with an exclusion zone of 4 samples) and the top $k = 2$ discords. Restricting the motif extraction to $k = 2$ prevents the identification of minor shifts of already-extracted patterns, thereby highlighting only distinct, independent daily routines.

The top panel of the figures displays the smoothed raw enmo time-series (measured in g) highlighting the motifs (blue/green) and discords (red). The bottom panel displays the corresponding Matrix Profile distance profile.

Subject 3846 (Figure 14) represents the low-risk medoid. The Matrix Profile analysis extracted two distinct motifs. The first motif captures a regular, decaying activity curve that represents the child settling down in a highly structured way at similar times across days. The second motif highlights a secondary repeating pattern, showing that the subject maintains a predictable structure in daily rest cycles. The top discord represents a short-lived burst of physical activity that temporarily breaks the sedentary routine, which is typical of healthy subjects taking active micro-breaks during sitting.

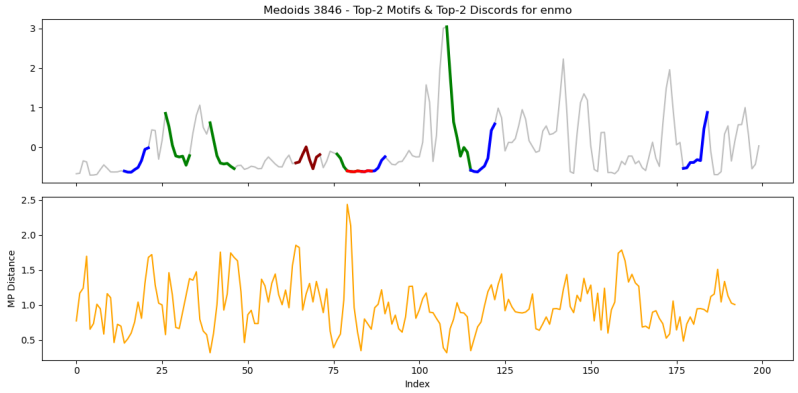


Figure 14: Matrix Profile analysis for Subject 3846 (Low-Risk Medoid)

In contrast, Subject 4386 (Figure 15) represents the high-risk medoid. Although it belongs to the same sedentary cluster, its local routines exhibit a different pattern. The first motif, while geometrically similar in shape, it shows a flatter trend with fewer micro-movements compared to the healthy subject. The second motif confirms a repeating sedentary block, the top discord corresponds to a prolonged period of extremely low physical activity (flat line). This pattern represents a long, continuous screen-time or sitting session without active breaks, highlighting the persistent sedentary behavior that is strongly correlated with problematic internet use.

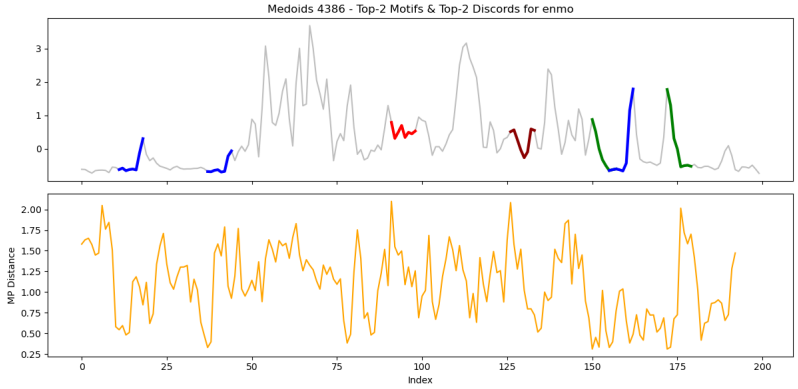


Figure 15: Matrix Profile analysis for Subject 4386 (High-Risk Medoid)

2.2.3 Comparison with Shapelets

To observe the relationship between local motifs and global discriminative shapelets, we selected the most important shapelet identified by our RDST + RandomForest classifier, ranked by the feature importance of the forest model. While motifs and discords were extracted on the univariate enmo channel, the RDST shapelets are inherently multivariate. For consistency, we project the most discriminative shapelet onto the enmo channel, obtaining a univariate view of the global high-risk signature. The length of this shapelet is $m = 12$ timesteps, which matches the window size of the motifs (6 hours of observation), which is highly comparable to the window size of the local motifs ($m = 8$, i.e., 4 hours), allowing a direct comparison at a similar temporal scale.
Crucially, this comparison is performed on two subjects belonging to the same sedentary cluster (Cluster 1). This provides a highly controlled context. The quantitative comparison reveals that for both medoid subjects, the enmo projection of the global risk shapelet does not align with their typical daily routines. Instead, it exhibits a high distance and strong negative correlation with their Top Motifs. The risk shapelet instead aligns closely with the Top Discords of both subjects. This suggests that for both subjects within this sedentary cluster, the flat, risk-associated behavior does not represent the dominant daily routine. Rather, it manifests as a local deviation or anomaly (a discord) from their main rest-activity patterns. The classifier's ability to distinguish between the High-Risk and Low-Risk subjects within the same sedentary baseline does not rely on a fundamental difference in their dominant routines (motifs), which remain structurally distinct from the shapelet.

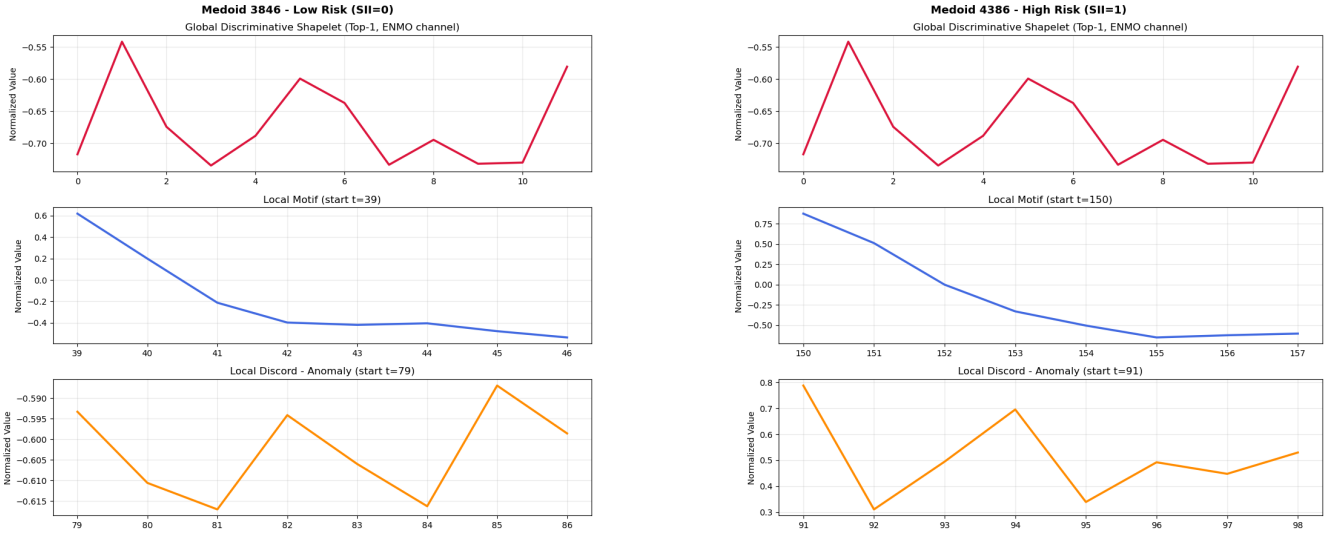


Figure 16: Global Shapelet vs. Local Motif and Discord

2.3 Time Series Clustering

In this section, the unsupervised clustering task applied to the time series dataset is presented, aiming to identify intrinsic patterns and natural groupings within the data. Two different non-supervised clustering algorithms were implemented and compared: K-Means and Agglomerative Hierarchical Clustering. To compute these algorithms, the PAA approximated dataset was used since utilizing the complete time series was too computationally expensive. Among the variables in the dataset, the ones utilized for the clustering were X, Y, Z, enmo, angle_z, and light. The discarded variables were subsequently used in the validation phase of the study to verify whether the identified clusters corresponded to meaningful groupings within these unused features.

The K-Means algorithm was implemented utilizing both the Euclidean distance and the Dynamic Time Warping (DTW) distance. A grid search was performed to find the optimal spatial constraint, while the number of clusters was determined using the elbow method and by observing the silhouette scores. The Agglomerative Hierarchical Clustering was instead implemented with just the Euclidean distance; the linkage type and the number of clusters were both derived from observing the corresponding dendrograms. A summary of the hyperparameters is shown in table 14.

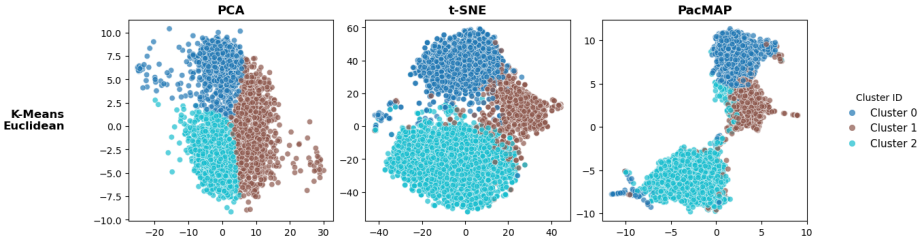
Table 14: Clustering algorithms parameters configuration

Modello	n_clusters	metric	init/linkage	n_init	other
K-Means Euclidean	3	euclidean	kmeans++	10	-
K-Means DTW	3	dtw	kmeans++	10	window* = 0.1
Hierarchical	3	euclidean	ward	-	-

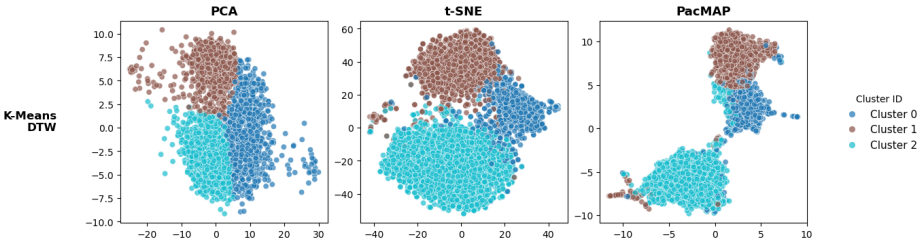
* The `window` parameter represents the Sakoe-Chiba band dimension.

2.3.1 Visualization

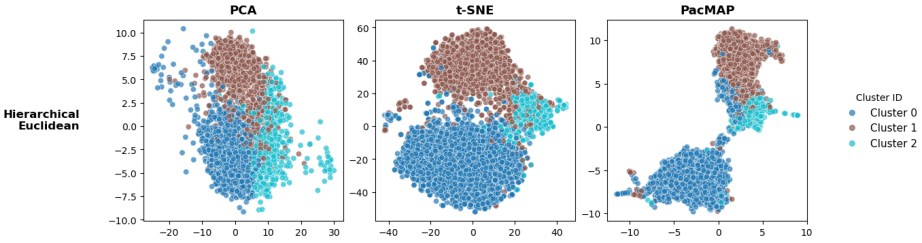
To visually inspect the cluster structures in a low-dimensional space, three dimensionality reduction techniques were adopted: PCA, t-SNE and PaCMAP. In Figure 17 the 2D projections are displayed. Both the groupings discovered by Kmeans Algorithm exhibit quite well separated groups, especially in the t-SNE and PaCMAP diagrams. By observing the image, it becomes clear how the kmeans algorithms divided the space in almost identical clusters, which implies that there is no significant time warping between the time series. The Hierarchical clustering, while being quite similar to the Kmeans counterparts, showed a greater amount of overlaps between groups. In Figure 18 it is possible to observe the partition inside the groups considering the omitted variables. Unfortunately, there is no clear pattern, as it seems every variable is evenly distributed between the clusters.



(a) K-Means Euclidean with PCA, t-SNE, and PaCMAP projections



(b) K-Means DTW with PCA, t-SNE, and PaCMAP projections



(c) Hierarchical Euclidean with PCA, t-SNE, and PaCMAP projections

Figure 17: Comprehensive comparison of clustering algorithms and dimensionality reduction techniques.

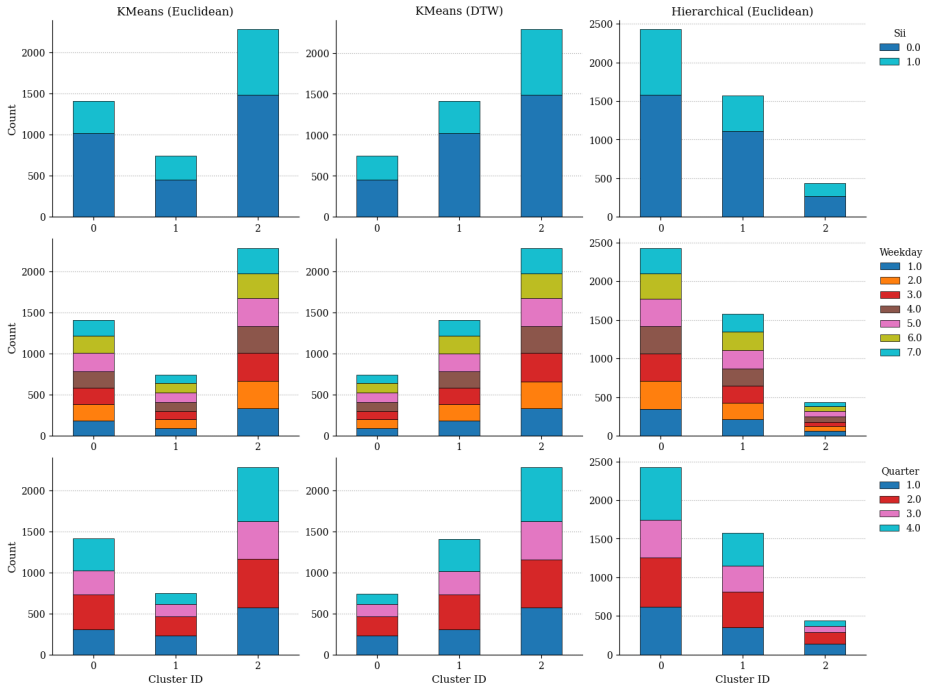


Figure 18: Distribution of unused variables within the clusters

2.3.2 Cluster Analysis

Finally, in Figure 19 the centroids of the clustering are shown. The features X, Z, and anglez are the ones that change the most between the groups, while enmo and light exhibit a nearly overlapping pattern across all clusters; this fact reveals that the segmentation performed by the algorithms is not driven by the absolute volume of physical activity, but rather by some specific postural habits. For the following description, the Kmeans DTW clustering was the one considered:

- **Cluster 0:** This group stands out for its positive and persistent values on the `Z` axis and for the arm inclination (`anglez`), while the `x` and `Y` axes remain close to zero. Such a kinematic configuration suggests the prolonged maintenance of a specific posture during the active phase of the day, possibly indicating the arm resting horizontally on a surface like a desk or table.
- **Cluster 1 and Cluster 2:** These two groups share a noticeably different baseline posture compared to Cluster 0. They both exhibit flat, negative values for `Z` and `anglez`. What clearly separates these two groups is entirely driven by the `x` axis, which behaves in exactly opposite ways: while Cluster 1 shows a broad positive surge during the active daytime hours, Cluster 2 experiences a symmetrical downward dip. In the context of the experiment, this mirrored pattern on a single axis might indicate a difference in sensor placement rather than actual human behavior. It is likely that these clusters do not represent distinct activity profiles, but are simply separating the children based on which arm they wore the tracker, either left or right, resulting in inverse spatial signals while performing the exact same everyday tasks.

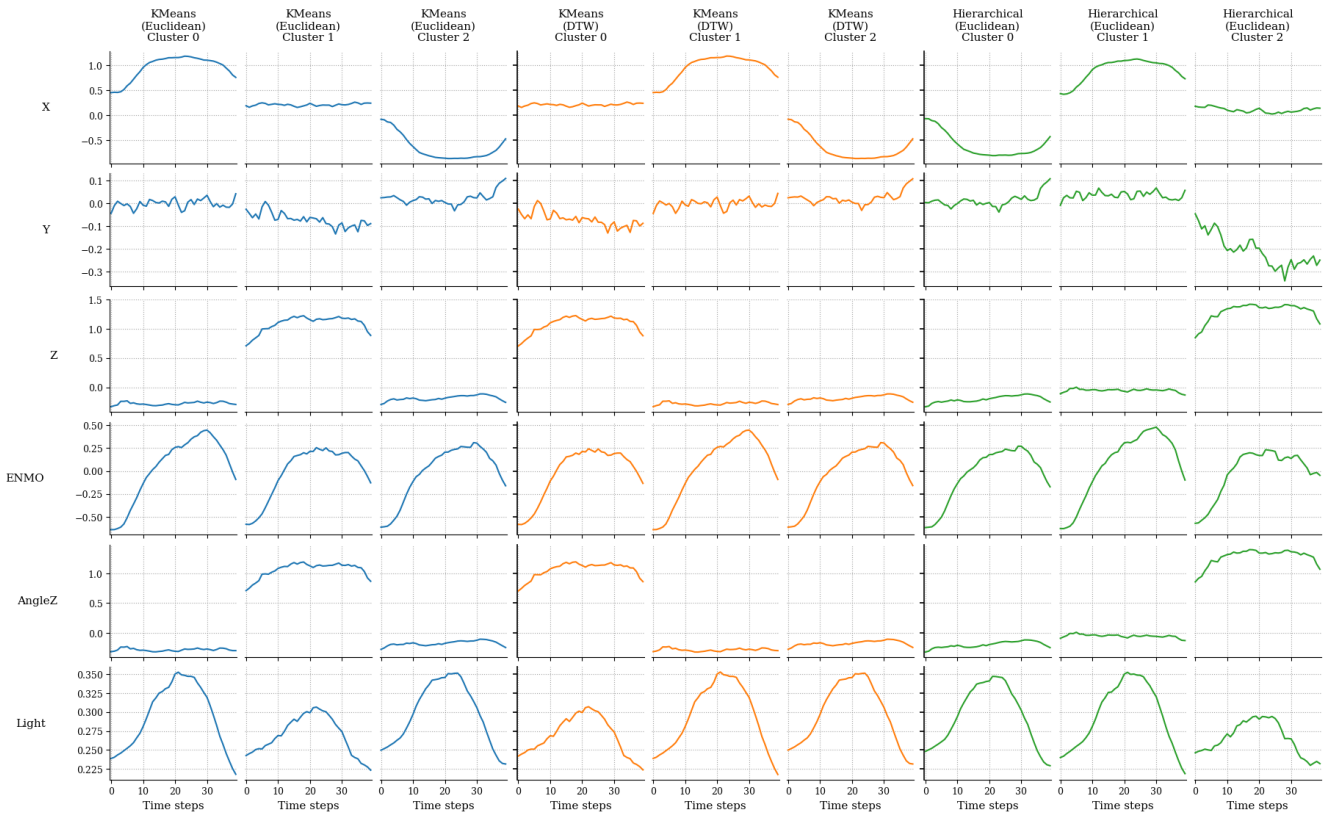


Figure 19: Cluster Centroids

2.3.3 Patient-Level Clustering

As it was anticipated in the chapter relative to the dataset preprocessing, some patients were associated with multiple time series. In this section, a clustering technique implemented to address this aspect of the dataset is discussed. To prevent patients with numerous recorded time series from biasing the cluster formations and to extract robust subject-level profiles, a Monte Carlo consensus clustering approach was developed.The procedure consisted of 100 iterations; in each iteration, a deterministic stratified sampling strategy was employed to randomly select exactly one time series per unique patient. This ensured that the clustering algorithm processed a perfectly balanced subset of data in every cycle, with each patient carrying equal weight. A Time Series K-Means algorithm (configured with Euclidean distance and $k = 2$) was subsequently applied to these subsamples directly on the 3D tensor representations. Across the 100 iterations, a consensus matrix was populated by tracking the co-occurrence frequency—specifically, the ratio of times any given pair of patients was assigned to the same cluster. Upon completion of the Monte Carlo simulation, this consensus matrix was transformed into a distance matrix by subtracting the consensus frequencies from one. A final hierarchical clustering step, utilizing average linkage, was then performed on this distance matrix to extract the definitive, stable patient-level clusters. Ultimately, to evaluate the separability and quality of these stable patient clusters, the final labels were mapped back to the entirety of the original time series. This allowed the global dataset to be projected and visually analyzed through dimensionality reduction techniques, namely PCA, t-SNE, and PacMAP. As seen in figure 20, the resulting clusters are apparently worse than the previously discussed ones

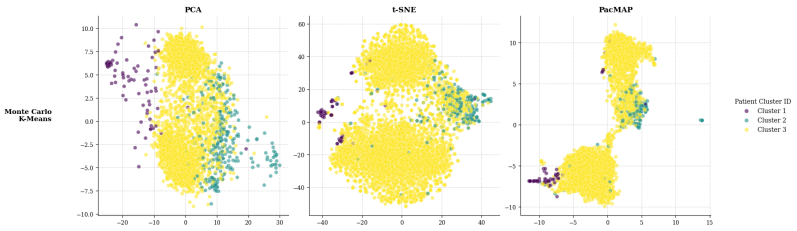


Figure 20: Patient-Level Clustering

2.3.4 Catch22

To further investigate the temporal dynamics of the dataset beyond raw sequence matching, a feature-based representation was adopted using the catch22 (canonical time-series characteristics) algorithm. This task utilized the denoised dataset and retained the previously selected variables. The catch22 transformation was applied to compute 22 distinct statistical, topological, and dynamical features for each attribute of every time series. To address missing values arising during the feature computation, a simple iterative imputation algorithm was employed to estimate and replace the missing data points. Subsequently, the feature space was standardized to ensure zero mean and unit variance across all dimensions. This preprocessed representation was then utilized to perform a secondary clustering analysis, comparing K-Means, Agglomerative Hierarchical clustering, DBSCAN. These models were evaluated both directly on the standardized catch22 features and on a reduced feature space obtained via Principal Component Analysis (PCA), set to retain 70% of the cumulative explained variance. In figure 21 are shown some of the clusters obtained from the standardized complete features; the numbers of clusters was chosen through the elbow method and dendrogram respectively for K-Means and hierarchical techniques, while, for DBSCAN, epsilon (18.5) and min_samples (10) were chosen through a grid search. As it was expected from the parameter tuning phase, there is no clear division in the dataset and a similar result was observed for the other methodologies. Consequently, this technique was discarded from the final discussion.

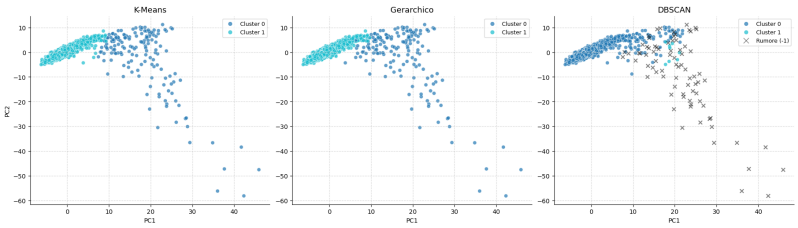


Figure 21: Clustering Catch22

2.3.5 Conclusions

In conclusion, every unsupervised clustering analysis on the full dataset partitioned the time series into three groups. The almost identical results found using both Euclidean and DTW distances indicate an absence of significant temporal shifts among the sequences. While the external variables evaluated did not show a strong correlation with the identified groupings, the centroid analysis highlighted distinct spatial signatures. Specifically, the clustering separated prolonged stationary desk-like postures from active behaviors, with the latter being further divided likely based on sensor placement (left versus right arm) rather than differing activity profiles. Since every technique produced a similar partitioning of the patients, there is no clear better methodology in this situation.

2.4 Time Series Classification

This section presents the time series classification analysis. The objective is to classify the sii_binary variable using X, Y, Z, enmo, anglez, and light as independent features. Three techniques are implemented: Shapelets, KNN and MiniRocket. The dataset chosen for the task is the unapproximated normalized one created during pre preprocessing phase. The dataset was split into a strictly stratified on sii_binary, with a 70% training and 30% testing partition.

2.4.1 Shapelets

Shapelets are local sub-sequences that maximize the statistical separation between our target classes. Unlike global alignment methods, shapelet-based classifiers identify localized, phase-independent patterns that are highly discriminative.

For the classification task, we utilized the RDSTClassifier pipeline, which integrates the Random Dilated Shapelet Transform (RDST) with a downstream tree-based ensemble. To reduce variance and prevent overfitting on the high-dimensional transformed feature space, a RandomForestClassifier was selected as the internal estimator.

Hyperparameter tuning was conducted using RandomizedSearchCV with 5-fold cross-validation, optimizing for the macro F1-score to account for the class imbalance. The search space optimized both the RDST extraction parameters and the internal Random Forest hyperparameters. The best estimator configuration resulted in Random Forest parameters: n_estimators = 200, max_depth = 5, min_samples_split = 10, min_samples_leaf = 16, max_features = 'sqrt', and criterion = 'gini', class_weight= 'balanced'. For the RDSTClassifier, on the other hand, the following parameters were used: max_shapelets = 1000, shapelet_lengths = $[4, 8, 12]$, and use_prime_dilations = True.

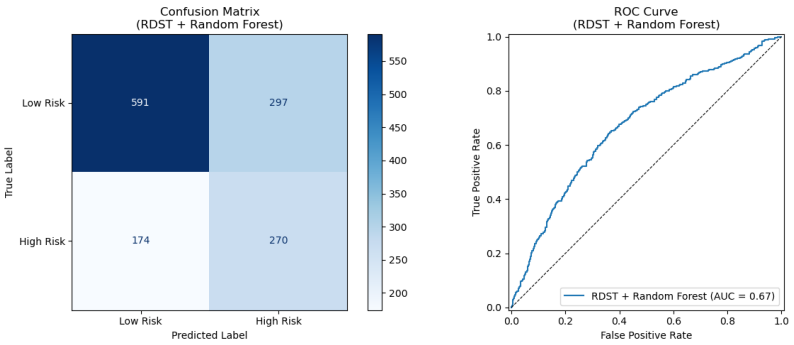


Figure 22: RDST + RandomForest Classification Performance

Class	Prec	Rec	F1	Supp
sii=0	0.77	0.67	0.72	888
sii=1	0.48	0.61	0.53	444

Table 15: Test Classification Report (Shapelets)

Under class imbalance, the recall remains balanced across both classes (0.67 for Low Risk and 0.61 for High Risk). The precision of 0.48 for the High-Risk class reflects a moderate false-positive rate, consistent with the behavioral overlap between the two groups.

To evaluate feature importance, we leveraged the Mean Decrease in Impurity (MDI) computed across the forest ensemble. Since the RandomForest aggregates predictions over 200 randomized trees, it provides a stable and globally consistent ranking of the features. On a global scale, the overall contribution of the three feature types is distributed as follows: Minimum Distance features account for 48.1% of the total importances, followed by Frequency (28.1%) and Temporal Position (23.7%).

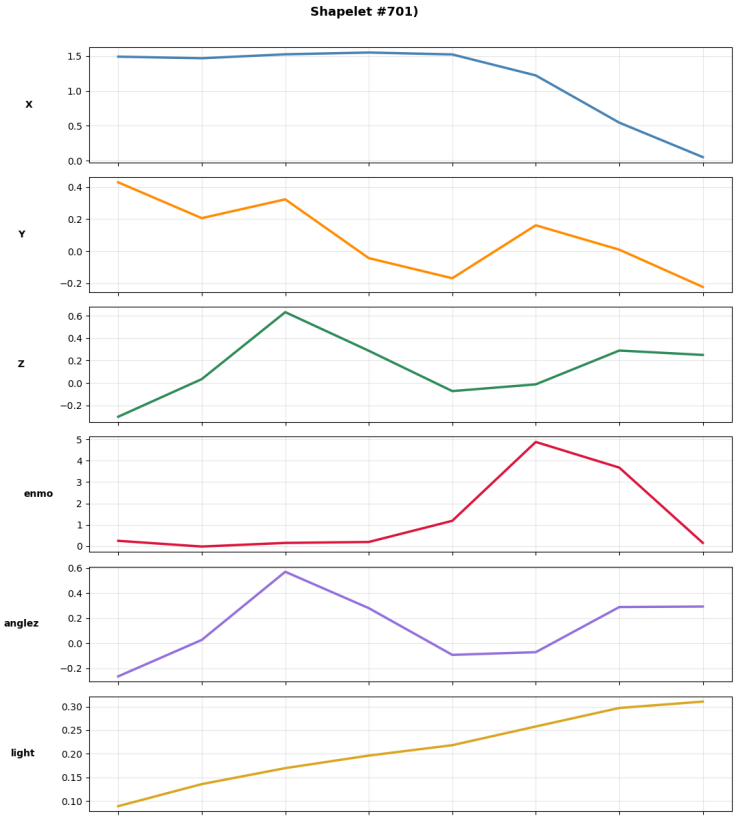


Figure 23: Most discriminative Shapelet (#701)

Figure 23 visualizes the most discriminative shapelet across all six dynamic channels (12 steps). Morphological inspection reveals a coherent, coordinated waveform profile across the accelerometer axes X, Y, and Z. This consistency is physically interpretable and the corresponding changes in total intensity (enmo) and body inclination (anglez) further confirm this, showing that the shapelet captures a coordinated behavioral transition rather than noise.

2.4.2 K-Nearest Neighbors

The K-Nearest Neighbors (KNN) algorithm is a non-parametric classification method that assigns a class label to an unseen instance based on the majority vote of its nearest neighbors in the training set. In this study, both Euclidean and DTW distance metrics were evaluated on the approximated dataset. To identify the optimal configurations, a Grid Search cross-validation was conducted, optimizing for the macro F1-score. The hyperparameter tuning determined $k = 1$ as the optimal number of neighbors for the Euclidean-based model. For the DTW-based model, the best performance was achieved with $k = 1$ and a warping window proportion of $w = 0.10$. The results are shown in Figures 24, 25 and Tables 16, 17. The results indicate that both KNN classifiers perform adequately in identifying instances of class 0, but struggle significantly to recognize observations where sii = 1. Although DTW provides a marginal improvement over the Euclidean distance in terms of overall accuracy and F1-score, the predictive power for class 1 remains weak.

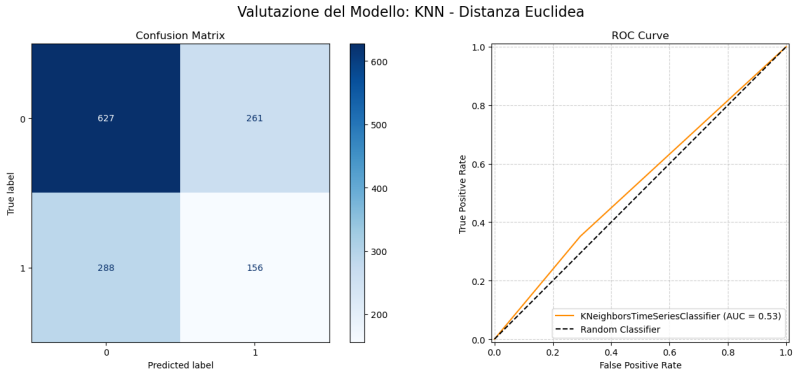


Figure 24: KNN Euclidean Classification

Class	Prec	Rec	F1	Supp
sii=0	0.69	0.71	0.70	888
sii=1	0.37	0.35	0.36	444
Macro avg	0.53	0.53	0.53	1332

Table 16: Classification Report (KNN Euclidean)

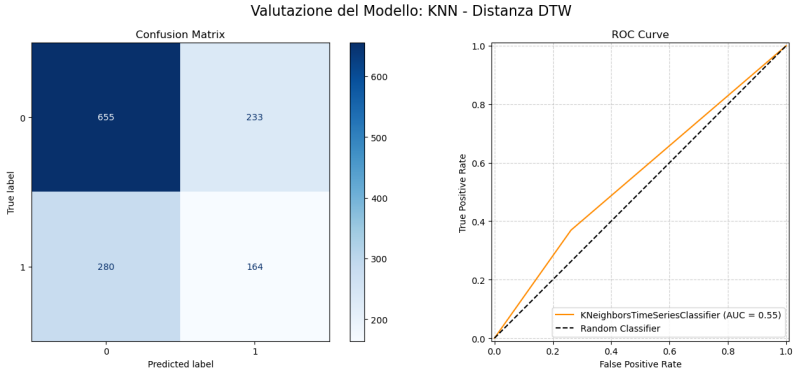


Figure 25: KNN DTW Classification

Class	Prec	Rec	F1	Supp
sii=0	0.70	0.74	0.72	888
sii=1	0.41	0.37	0.39	444
Macro avg	0.56	0.56	0.56	1332

Table 17: Classification Report (KNN DTW)

2.4.3 MiniRocket

Finally, the last classification method employed in this study is MiniRocket. Based on the architecture of ROCKET (RandOm Convolutional KErnel Transform), MiniRocket improves computational efficiency by using a small, fixed, and deterministic set of kernels. By applying these kernels to the input time series and extracting a single pooling feature, the algorithm captures essential temporal characteristics while significantly reducing computational overhead. The extracted features are subsequently classified using a linear model with Ridge (L2) regularization. While the primary hyperparameter is the total number of features (kernels), a Grid Search cross-validation showed that the default configuration provided the best performance.; consequently, the default parameters were retained; the lambda parameter for the Ridge Classification is automatically calculated by the algorithm itself. The results are presented in Figure 26 and Table 18. While the model performs adequately in identifying and classifying instances of class 0, it struggles to recognize observations where sii = 1, resulting in an overall performance that is only marginally better than a random classifier.

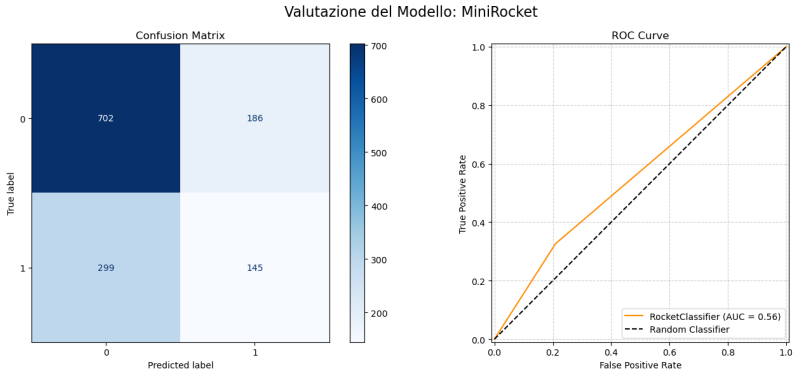


Figure 26: Minirocket Classification

Class	Prec	Rec	F1	Supp
sii=0	0.70	0.79	0.74	888
sii=1	0.44	0.33	0.37	444
Macro avg	0.57	0.56	0.56	1332

Table 18: Classification Report

2.4.4 Conclusions

The comparative analysis of the implemented time series classification methodologies reveals that the Shapelet-based approach achieved the most robust predictive performance. As demonstrated by the macro average measurements, it outperformed the alternative models, recording a macro F1-score of 0.62, compared to 0.56 for both KNN DTW and MiniRocket, and 0.53 for KNN Euclidean. This advantage is further evidenced by the Receiver Operating Characteristic (ROC) curve analysis; the Shapelet method attained an AUC of 0.67, significantly outperforming the other techniques, none of which exceeded 0.56.

A critical limitation observed across all evaluated algorithms is the inherent difficulty in correctly classifying instances belonging to class 1. While KNN and MiniRocket yielded adequate results for the majority class (sii=0), their predictive power for the minority class remained weak, frequently performing only marginally better than a random classifier. The Shapelets methodology proved to be the most effective solution in mitigating this issue. It offered a significantly better balance in recognizing both classes, achieving a higher F1-score (0.53) for the critical sii=1 category compared to the other techniques, which ranged between 0.36 and 0.39. Consequently, RDST stands out as the most suitable methodology among those tested for this specific classification task.